Abdelrahman Khalifa

Software Engineer | Rust, C, C++, Linux, Embedded

abdelrahman.khalifa.gad@gmail.com | +20 114 250 5042 | Cairo, Egypt

linkedin.com/in/abd002 | github.com/Abd002 | codeforces.com/profile/_Khalifa

SUMMARY

Software engineer working in Rust, C and modern C++ on Linux and embedded systems. At EKSON, builds the Zephyr firmware, PyQt5 desktop app and Node.js backend of its motion simulators; built the Printers page for COSMIC Settings as a Google Summer of Code 2026 contributor with The Linux Foundation (OpenPrinting), with 7 PRs merged upstream in cups-rs and libcups, plus accepted contributions to Zephyr RTOS, Boa and Rust Clippy. Codeforces Expert and 4-time ECPC finalist.

EXPERIENCE

Google Summer of Code Contributor | **The Linux Foundation (OpenPrinting)** | **May 2026 – Oct 2026**

Open-source project: Printers page for COSMIC Settings, in Rust; passed the final evaluation, still maintaining it; mentored by OpenPrinting and System76 | Google Summer of Code project page | github.com/Abd002/GSOC-2026 | github.com/Abd002/cosmic-printers

- Built the **Printers page for COSMIC Settings** and a standalone printers app that share the same UI: printers grouped by device, state and supply levels, print queue control (pause, resume, cancel, move jobs), default printer and options, test pages and live updates (cosmic-settings #2182, in review).
- Implemented the Add Printer flow: legacy and specialty printers are set up through Printer Applications (PAPPL) instead of PPD drivers, following the CUPS 3 architecture.
- Designed the stack as 5 Rust crates: `printers-core` (types, grouping queues into physical devices), `printers-client` (async varlink client), `printers-server` (CUPS, IPP, DNS-SD and Printer Application backend), `printers-ui` (iced/libcosmic) and `printers-app`.
- Exposed the backend over varlink IPC from `cosmic-settings-daemon` (cosmic-settings-daemon #188, in review), so the C FFI code never loads in the Settings process; the standalone app can run the server in-process.
- Got **7 PRs merged upstream** in OpenPrinting. cups-rs (Rust): ported it to libcups3 with CUPS 2 and CUPS 3 builds and CI; added IPP management operations, Printer Application support, IP/port connections, `lpoptions` and FFI bindings for the libcups3 DNS-SD API (PRs #18–#22). libcups (C): fixed crashes in `cupsDNSSDDelete()` when Avahi fails to initialize, and a leaked client in the Avahi DNS-SD code (#164, #165).
- Tested on real HP printers and Printer Applications (Ghostscript, Gutenprint, HPLIP, LPrint); CI checks formatting, Clippy and tests on several architectures and runs CUPS, Avahi and `ippeveprinter` to test discovery.

Embedded Software Engineer | **EKSON** | **Aug 2025 – Present**

Software for 3-DoF and 6-DoF pneumatic motion simulators: firmware, desktop app, backend and CI; part-time, remote

- Wrote the ESP32-S3 firmware and migrated it from ESP-IDF/FreeRTOS to Zephyr RTOS for more deterministic timing, devicetree-based drivers and easier porting to other boards.
- Implemented closed-loop PID control of the pneumatic valves, with feedback from an MPU6050 IMU over I2C (calibration and a Mahony filter for orientation), safety stops and a watchdog; UART protocol to the PC.
- Built the PyQt5 desktop control app; cut its CPU usage and latency by about 40% (measured) by using asyncio to wait for ESP32 serial and server messages concurrently, with processing in background threads, off the UI thread.
- Engineered remote operation: a web app and a Node.js/Express + MongoDB backend that the desktop app polls over REST, relaying commands to the simulator over UART.
- Added firmware updates: new firmware uploaded on the web is downloaded by the desktop app and flashed to the ESP32-S3 with esptool; GitHub Actions CI for builds and tests.
- Developed a Unity proof of concept that streams game motion to the simulator.

Embedded Linux Systems Intern | **Orange Digital Center (ODC)** | **Dec 2024 – Jan 2025**

- Created custom Linux images with Yocto and Buildroot and wrote a Linux kernel module; worked with device trees, root file systems and hardware flashing. Scored 100% in the final evaluation.

OPEN SOURCE PROJECTS

- **Boa, a JavaScript engine in Rust** (Dec 2025): optimized `JsStr::code_points()` for Latin1 strings, avoiding UTF-16 conversion; with tests (PR #4553, landed upstream).
- **Rust Clippy** (Dec 2025): fixed `ref_as_ptr` false positives for non-temporary references in `let/const` (PR #16214, accepted).
- **Zephyr RTOS** (Mar 2025, first open-source contribution): implemented the `IP_MULTICAST_LOOP` socket option for IPv4 across 6 networking modules, reviewed by the networking maintainers and merged upstream (issue #86977, PR #87167, commit b117036).

PROJECTS

- **AUTOSAR-Based V2X Ecosystem for Software-Defined Vehicles** (graduation project, team; STM32, FreeRTOS, AUTOSAR): owned the firmware update path (an ESP32/Raspberry Pi gateway sends the image to the STM32 over UART, after a first attempt over CAN) and a dual-bank bootloader that checks the CRC before jumping and rolls back on failure. Also studied the Trampoline OSEK/VDX RTOS and helped the team start on it before it moved to FreeRTOS. GitHub

- **IoT Device Communication System** (C++, Raspberry Pi 4, QEMU, Yocto): modular object-oriented framework for TCP unicast and UDP multicast between a Pi server and emulated clients; Yocto cross-compilation and Make. GitHub
- **Kernel Monitor** (C, Linux, ARM, QEMU): character device driver that walks the process list under RCU and exports per-task IDs, memory, threads, context switches and CPU time to user space; run on QEMU. GitHub
- **Seat Heater Control System** (Tiva C, FreeRTOS): real-time controller with temperature monitoring, heater-level control and UART diagnostics, using tasks, mutexes and event flags. GitHub
- **AUTOSAR PORT and GPIO drivers** (C, TM4C): developed and validated MCAL drivers following the AUTOSAR specification.
- **Traffic Light Controller** (Verilog, Spartan-3E FPGA): FSM with debounce and delay modules. GitHub
- **Door Locker Security System** (C, two MCUs): password lock with keypad, LCD, I2C EEPROM, timer and DC motor; UART between the MCUs. GitHub
- **Operating System Simulator** (Java): CLI that simulates process scheduling, memory management, system calls and locks. GitHub
- **Function Solver and Plotter** (Python, PySide2, SymPy, Matplotlib): GUI that solves and plots two functions with intersection points; validation, error handling and end-to-end tests with pytest-qt. GitHub
- **Inventory Management System** (C#, WinForms, SQL, Telerik UI, Crystal Reports): stock, orders and deliveries with role-based access control, automated reports and local databases for offline use. GitHub
- **Sports Management Web App** (team; ASP.NET Core MVC, C#, MS SQL, Entity Framework): role-based CRUD dashboards; designed the relational schema and wrote the backend and account-management logic. GitHub
- **Phonebook** (React, Node.js, Express, MongoDB): full-stack app with a REST API. GitHub
- **Team projects:** Sarh Al-Tafawuq, a production web platform (live); Nafaqati, a Flutter Android expense tracker with on-device SMS import, charts and offline storage (Google Play).
- **Other projects:** Children Safety web app (Flask, geolocation; GitHub), desktop chess in C# (GitHub), face-mask detection with a CNN, and a smart-garage IoT system deployed with Yocto on Intel Galileo.

TECHNICAL SKILLS

Languages: Rust, C, Modern C++, Python (asyncio), Java, C#, SQL, JavaScript, Bash, Verilog/VHDL, MATLAB
Systems & Linux: Linux (Ubuntu/Debian), operating systems, FFI, IPC (varlink, D-Bus; zbus, zlink), async/event-driven design, multithreading, memory management, CUPS/IPP, DNS-SD/mDNS (Avahi), kernel modules and device drivers
Embedded: Zephyr (devicetree, west), FreeRTOS, ESP-IDF, OSEK/VDX (Trampoline), ARM Cortex-M (STM32, TM4C), ESP32-S3, AVR, AUTOSAR, bootloaders, FOTA, PID control, Yocto, Buildroot, QEMU; UART, SPI, I2C, CAN
Tools & practices: Git/GitHub, Cargo, Clippy, CMake, Make, GitHub Actions CI, esptool, unit and end-to-end testing (pytest, pytest-qt), debugging, code review, Xilinx FPGA tools, PCB design
Web, desktop & ML: React, Node.js, Express, MongoDB, REST APIs, ASP.NET Core, Flask; iced, libcosmic, PyQt5, WinForms; TensorFlow, PyTorch

ACHIEVEMENTS

- Codeforces Expert (handle _Khalifa); 4-time ECPC finalist, 2021–2024 (Egyptian Collegiate Programming Contest).
- Treasurer, IEEE Student Branch, Minia University: managed a budget of more than $10k.

COURSES

Deep Learning (CNNs, TensorFlow/PyTorch) · Machine Learning Basics · RTOS Programming (FreeRTOS, OSEK/VDX, AUTOSAR OS) · Embedded Automotive (AUTOSAR, CAN/LIN, ISO 26262) · ARM Architecture (Cortex-M, drivers, DMA) · Embedded Systems Diploma (AVR, RTOS) · Full Stack Open (React, Node.js, MongoDB)

EDUCATION

Bachelor of Science (B.Sc.), Computer and Systems Engineering | **Minia University** | **2020 – 2025**

- Grade: Very Good with Honors.